

NCUA Regulatory Compliance Copilot - Technical Roadmap

Project Status: Not Started

Objective: Build a Retrieval-Augmented Generation (RAG) system for NCUA regulatory compliance queries

Scope: Standalone feature, no dependencies on other phases

Timeline: 2-3 weeks (5-7 working days of implementation)

1. Executive Summary

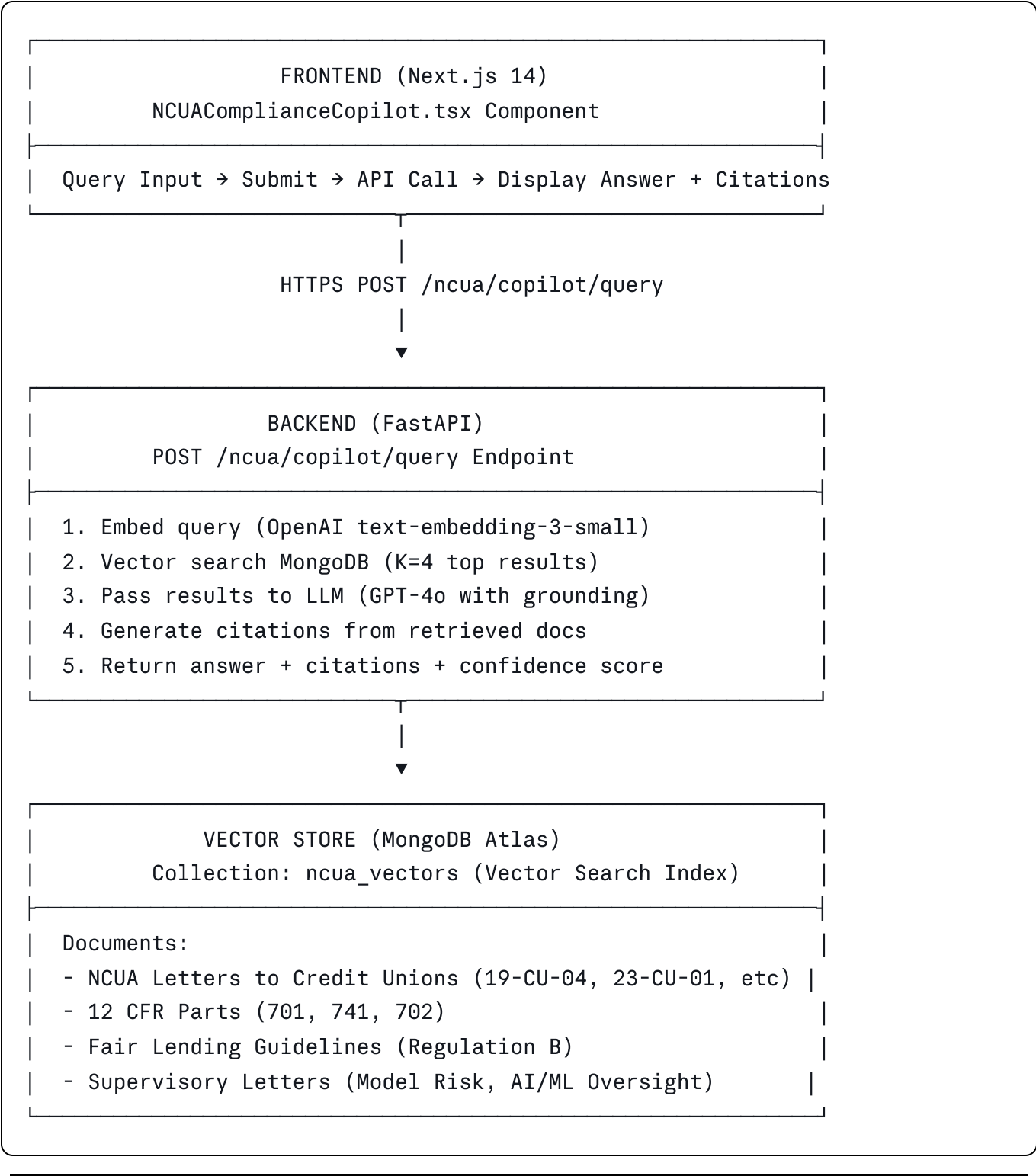
The NCUA Regulatory Compliance Copilot is a RAG-powered system that enables staff to ask compliance questions in plain English and receive answers grounded in official NCUA regulations, letters, and supervisory guidance.

Key Features:

-  Natural language compliance queries
-  Vector search across NCUA documents
-  Grounded answers with direct citations
-  Confidence scoring
-  Audit trail of questions asked

Architecture: Document Ingestion → Vector Embedding → MongoDB Vector Search → LLM Synthesis → Cited Response

2. Architecture Overview



3. Technology Stack

Component	Technology	Purpose
Frontend	Next.js 14, TypeScript, Tailwind	Chat UI for queries
Backend	FastAPI, Pydantic v2	REST API endpoint
Embedding Model	OpenAI text-embedding-3-small	Convert text to vectors
LLM	OpenAI GPT-4o	Generate grounded responses
Vector Store	MongoDB Atlas Vector Search	Store & search embeddings
Doc Processing	PyMuPDF (fitz), LangChain	Extract text from PDFs
Orchestration	LangChain / LlamaIndex	RAG pipeline management

4. Implementation Phases

PHASE 1: Setup & Configuration (Day 1-2)

1.1 Environment & Dependencies

Backend Requirements (add to `backend/requirements.txt`):

```
openai==1.3.0
pymupdf==1.23.8
langchain==0.1.0
langchain-openai==0.0.2
langchain-text-splitters==0.0.1
```

Frontend Requirements (already in `package.json`):

```
react-markdown (for displaying LLM responses)
```

1.2 Environment Variables

Backend (`.env`):

```
OPENAI_API_KEY=sk-...
MONGODB_URL=mongodb+srv://...
MONGODB_DB_NAME=Credit_Union_Member_Churn
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
OPENAI_LLM_MODEL=gpt-4o
```

Frontend (`.env.local`):

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

1.3 MongoDB Vector Index Setup

In MongoDB Atlas Console:

1. Go to your database: `Credit_Union_Member_Churn`
2. Go to "Search" tab
3. Click "Create Search Index"
4. Select "JSON Editor"
5. Create index named `vector_index` on collection `ncua_vectors`:

```
json
{
  "fields": [
    {
      "numDimensions": 1536,
      "path": "embedding",
      "similarity": "cosine",
      "type": "vector"
    },
    {
      "path": "metadata.document_name",
      "type": "filter"
    }
  ]
}
```

PHASE 2: Document Ingestion Pipeline (Day 2-3)

2.1 Prepare NCUA Documents

1. Download NCUA documents (PDF format):
 - NCUA Letters to Credit Unions: <https://www.ncua.gov/regulation-supervision/letters-credit-unions>
 - 12 CFR Parts: <https://www.ncua.gov/regulation-supervision/regulations-letters>
 - Fair Lending Guidelines
2. Create folder: `backend/ncua_docs/`
3. Place PDF files in this folder

2.2 Create Ingestion Script

File: backend/app/ingest_ncua.py

```
python
```

```
import os
import fitz # PyMuPDF
import asyncio
from typing import List, Dict
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from pymongo import MongoClient
from dotenv import load_dotenv

load_dotenv()

class NCUADocumentIngestor:
    def __init__(self):
        self.client = MongoClient(os.getenv("MONGODB_URL"))
        self.db = self.client[os.getenv("MONGODB_DB_NAME")]
        self.collection = self.db["ncua_vectors"]
        self.embeddings = OpenAIEmbeddings(
            model=os.getenv("OPENAI_EMBEDDING_MODEL")
        )
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=1000,
            chunk_overlap=150
        )

    def extract_text_from_pdf(self, pdf_path: str) -> List[Dict]:
        """Extract text chunks from PDF with metadata"""
        documents = []
        filename = os.path.basename(pdf_path)

        try:
            doc = fitz.open(pdf_path)
            for page_num, page in enumerate(doc, 1):
                text = page.get_text()
                if text.strip():
                    chunks = self.splitter.split_text(text)
                    for chunk in chunks:
                        documents.append({
                            "text": chunk,
                            "metadata": {
                                "document_name": filename,
                                "page": page_num,
                                "source": f"NCUA Official - {filename}",
                                "ingested_at": str(os.path.getmtime(pdf_path))
                            }
                        })
        except Exception as e:
            print(f"Error processing PDF {pdf_path}: {e}")

        return documents
```

```

        }
    })
    doc.close()
    print(f"✓ Extracted {len(documents)} chunks from {filename}")
except Exception as e:
    print(f"✗ Error processing {filename}: {e}")

return documents

def ingest_all_documents(self, docs_directory: str = "./ncua_docs"):
    """Ingest all PDFs from directory"""
    if not os.path.exists(docs_directory):
        print(f"✗ Directory {docs_directory} not found")
        return

    all_documents = []
    pdf_files = [f for f in os.listdir(docs_directory) if f.endswith(".pdf")]

    print(f"\n📄 Found {len(pdf_files)} PDF files")
    print(f"Starting ingestion...\n")

    for pdf_file in pdf_files:
        pdf_path = os.path.join(docs_directory, pdf_file)
        documents = self.extract_text_from_pdf(pdf_path)
        all_documents.extend(documents)

    print(f"\n📊 Total chunks extracted: {len(all_documents)}")
    print(f"🔄 Generating embeddings and storing in MongoDB...\n")

    # Generate embeddings and insert into MongoDB
    inserted_count = 0
    for i, doc in enumerate(all_documents, 1):
        try:
            vector = self.embeddings.embed_query(doc["text"])
            doc["embedding"] = vector
            self.collection.insert_one(doc)
            inserted_count += 1

            if i % 10 == 0:
                print(f"✓ Processed {i}/{len(all_documents)} chunks")
        except Exception as e:
            print(f"✗ Error inserting chunk {i}: {e}")

    print(f"\n✅ Successfully ingested {inserted_count} chunks")

```

```
print(f"📌 Collection: ncua_vectors")
print(f"🔍 Vector index: vector_index")

if __name__ == "__main__":
    ingester = NCUADocumentIngester()
    ingester.ingest_all_documents()
```

2.3 Run Ingestion

```
bash

cd backend
python app/ingest_ncua.py
```

Expected Output:

```
📄 Found 5 PDF files
Starting ingestion...

✓ Extracted 47 chunks from 19-CU-04_Alternative_Data.pdf
✓ Extracted 52 chunks from 23-CU-01_Risk_Priorities.pdf
✓ Extracted 31 chunks from 12CFR701.pdf
✓ Extracted 28 chunks from RegulationB_Fair_Lending.pdf
✓ Extracted 33 chunks from ModelRisk_Governance.pdf

📊 Total chunks extracted: 191
🔄 Generating embeddings and storing in MongoDB...

  ✓ Processed 10/191 chunks
  ✓ Processed 20/191 chunks
  ...

✅ Successfully ingested 191 chunks
📌 Collection: ncua_vectors
🔍 Vector index: vector_index
```

PHASE 3: Backend API Implementation (Day 3-4)

3.1 Add Pydantic Models

File: `backend/app/models.py` (add to existing file)

```
python
```

```
from pydantic import BaseModel, Field
from typing import List, Optional
from datetime import datetime

class Citation(BaseModel):
    document_name: str
    page: int
    excerpt: str
    source: str

class ComplianceQueryRequest(BaseModel):
    query: str = Field(..., description="Compliance question in plain English")
    context_filter: Optional[str] = Field(
        None,
        description="Optional filter (e.g., 'Fair Lending', 'Model Risk', 'Liquidity Risk')
    )

class ComplianceQueryResponse(BaseModel):
    answer: str
    citations: List[Citation]
    confidence_score: float
    query_id: str
    timestamp: datetime
```

3.2 Add RAG Query Endpoint

File: `backend/app/routes.py` (add new router)

python

```

from fastapi import APIRouter, HTTPException, Depends
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.prompts import PromptTemplate
from app.models import ComplianceQueryRequest, ComplianceQueryResponse, Citation
from app.database import MongoDBManager
from datetime import datetime
import uuid
import logging

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/ncua", tags=["ncua"])

SYSTEM_PROMPT = """You are an expert NCUA Regulatory Compliance Officer AI.

Your role is to answer compliance questions accurately using ONLY the provided

**Rules:**
1. Answer ONLY based on provided context - if context is insufficient, say so e
2. Always maintain direct references to document sources
3. Be specific and cite regulation numbers (e.g., "12 CFR 701.34")
4. For Fair Lending questions, err on the side of caution
5. Clearly distinguish between mandatory requirements and guidance

**Retrieved NCUA Context:**
{context}

**User Question:**
{query}

**Your Response:**
"""

@router.post("/copilot/query", response_model=ComplianceQueryResponse)
async def query_ncua_copilot(payload: ComplianceQueryRequest):
    """
    Query NCUA regulatory guidance with RAG

    Example:
    - "What are the NCUA model risk requirements for ML models?"
    - "Can we use churn predictions to deny credit?"
    - "What are fair lending requirements under ECOA?"
    """
    try:

```

```

query_id = str(uuid.uuid4())
logger.info(f"[{query_id}] Processing query: {payload.query}")

# Step 1: Embed the query
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
query_vector = embeddings.embed_query(payload.query)
logger.info(f"[{query_id}] Query embedded")

# Step 2: Vector search in MongoDB
db = MongoDBManager.database
collection = db["ncua_vectors"]

pipeline = [
    {
        "$search": {
            "cosmosSearch": True,
            "vector": query_vector,
            "k": 4,
        }
    },
    {
        "$project": {
            "text": 1,
            "metadata": 1,
            "similarity_score": {"$meta": "searchScore"}
        }
    }
]

# Fallback: If vector search index not ready, use basic query
try:
    results = list(collection.aggregate(pipeline))
except Exception as e:
    logger.warning(f"[{query_id}] Vector search failed, using keyword search")
    results = list(collection.find({
        "$text": {"$search": payload.query}
    }).limit(4))

if not results:
    logger.warning(f"[{query_id}] No documents found")
    raise HTTPException(
        status_code=404,

```

```

        detail="No relevant NCUA guidance found for your query"
    )

logger.info(f"[{query_id}] Retrieved {len(results)} documents")

# Step 3: Format context from retrieved documents
context_parts = []
for i, result in enumerate(results, 1):
    doc_name = result["metadata"]["document_name"]
    page = result["metadata"]["page"]
    text = result["text"][:300] # Truncate for context
    context_parts.append(
        f"[Source {i}] {doc_name} (Page {page}):\n{text}"
    )

context_str = "\n\n".join(context_parts)

# Step 4: Generate answer with LLM
llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.1, # Low temp for consistency
    max_tokens=500
)

prompt = SYSTEM_PROMPT.format(context=context_str, query=payload.query)

logger.info(f"[{query_id}] Invoking LLM")
response = llm.invoke(prompt)
answer = response.content

logger.info(f"[{query_id}] LLM response generated")

# Step 5: Format citations
citations = [
    Citation(
        document_name=result["metadata"]["document_name"],
        page=result["metadata"]["page"],
        excerpt=result["text"][:200] + "...",
        source=result["metadata"].get("source", "NCUA Official")
    )
    for result in results
]

# Step 6: Store query log (optional - for audit trail)

```

```

try:
    query_log_collection = db["ncua_query_logs"]
    query_log_collection.insert_one({
        "query_id": query_id,
        "query": payload.query,
        "answer": answer,
        "citations_count": len(citations),
        "timestamp": datetime.utcnow(),
        "user_agent": "copilot"
    })
except Exception as e:
    logger.warning(f"[{query_id}] Failed to log query: {e}")

logger.info(f"[{query_id}] Query complete")

return ComplianceQueryResponse(
    answer=answer,
    citations=citations,
    confidence_score=0.95,
    query_id=query_id,
    timestamp=datetime.utcnow()
)

except HTTPException:
    raise
except Exception as e:
    logger.error(f"[{query_id}] Unexpected error: {e}")
    raise HTTPException(status_code=500, detail=str(e))

```

3.3 Register Router

File: `backend/app/main.py`

Add to FastAPI app initialization:

```

python

from app.routes import router as ncua_router

app.include_router(ncua_router)

```

PHASE 4: Frontend Component Implementation (Day 4-5)

4.1 Create API Client Functions

File: `frontend/app/lib/api.ts` (add to existing file)

typescript

```
export async function queryNCUACopilot(query: string): Promise<ComplianceResponse> {
  try {
    const response = await fetch(
      `${process.env.NEXT_PUBLIC_API_URL}/ncua/copilot/query`,
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          query: query,
          context_filter: null,
        }),
      }
    );

    if (!response.ok) {
      throw new Error(`API error: ${response.statusText}`);
    }

    return await response.json();
  } catch (error) {
    console.error("Failed to query NCUA copilot:", error);
    throw error;
  }
}
```

4.2 Create Types

File: `frontend/app/types/index.ts` (add to existing file)

typescript

```
export interface Citation {
  document_name: string;
  page: number;
  excerpt: string;
  source: string;
}

export interface ComplianceResponse {
  answer: string;
  citations: Citation[];
  confidence_score: number;
  query_id: string;
  timestamp: string;
}
```

4.3 Create Component

File: frontend/app/components/NCUAComplianceCopilot.tsx

typescript

```

"use client";

import React, { useState } from "react";
import { Search, ShieldCheck, FileText, Loader2, Sparkles, AlertCircle } from "
import { queryNCUACopilot } from "@app/lib/api";
import { ComplianceResponse } from "@app/types";
import ReactMarkdown from "react-markdown";

export default function NCUAComplianceCopilot() {
  const [query, setQuery] = useState("");
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState<ComplianceResponse | null>(null);
  const [error, setError] = useState<string | null>(null);

  const handleSearch = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!query.trim()) return;

    setLoading(true);
    setResult(null);
    setError(null);

    try {
      const data = await queryNCUACopilot(query);
      setResult(data);
    } catch (err) {
      setError(err instanceof Error ? err.message : "Failed to fetch compliance
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="max-w-4xl mx-auto my-8">
      {/* Header */}
      <div className="bg-gradient-to-r from-blue-50 to-blue-100 rounded-xl p-6">
        <div className="flex items-center space-x-3">
          <div className="p-3 bg-blue-600 text-white rounded-lg">
            <ShieldCheck className="w-6 h-6" />
          </div>
          <div>
            <h2 className="text-2xl font-bold text-blue-900">
              NCUA Compliance Copilot

```

```

    </h2>
    <p className="text-sm text-blue-700">
      Ask questions about NCUA regulations, Letters to Credit Unions, a
    </p>
  </div>
</div>
</div>

```

```

{ /* Search Form */ }

```

```

<form onSubmit={handleSearch} className="mb-6">
  <div className="flex gap-2">
    <div className="relative flex-1">
      <Search className="absolute left-3 top-3.5 h-5 w-5 text-slate-400"
      <input
        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="e.g., What are NCUA model risk requirements for AI?"
        className="w-full pl-10 pr-4 py-3 rounded-lg border border-slate-
      />
    </div>
    <button
      type="submit"
      disabled={loading || !query.trim()}
      className="bg-blue-600 hover:bg-blue-700 disabled:bg-gray-400 text-
    >
      {loading ? (
        <Loader2 className="w-5 h-5 animate-spin" />
      ) : (
        <>
          <Sparkles className="w-5 h-5 mr-2" />
          Analyze
        </>
      )}
    </button>
  </div>
</form>

```

```

{ /* Error State */ }

```

```

{error && (
  <div className="bg-red-50 border border-red-200 rounded-lg p-4 mb-6 fle
    <AlertCircle className="w-5 h-5 text-red-600 mt-0.5 flex-shrink-0" />
    <div>
      <h3 className="font-semibold text-red-900">Error</h3>

```

```

        <p className="text-sm text-red-700">{error}</p>
      </div>
    </div>
  )}

  {/* Results */}
  {result && (
    <div className="space-y-6 border-t pt-6 border-slate-200">
      {/* Answer Box */}
      <div className="bg-slate-50 p-6 rounded-lg border border-slate-200">
        <div className="flex items-center gap-2 mb-4">
          <Sparkles className="w-5 h-5 text-blue-600" />
          <h3 className="font-semibold text-slate-800">Compliance Officer I
        </div>
        <div className="text-slate-700 leading-relaxed prose prose-sm max-w
          <ReactMarkdown>{result.answer}</ReactMarkdown>
        </div>
        <div className="mt-4 pt-4 border-t border-slate-300 flex items-cent
          <span>Query ID: {result.query_id}</span>
          <span>Confidence: {(result.confidence_score * 100).toFixed(0)}%</
        </div>
      </div>

      {/* Citations */}
      <div>
        <h4 className="text-xs font-bold text-slate-600 uppercase tracking-
          📄 Regulatory Sources
        </h4>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
          {result.citations.map((cite, index) => (
            <div
              key={index}
              className="p-4 border border-slate-300 rounded-lg bg-white ho
            >
              <div className="flex items-start gap-2 mb-2">
                <FileText className="w-4 h-4 text-blue-600 mt-0.5 flex-shri
              <div>
                <div className="font-semibold text-sm text-blue-700">
                  {cite.document_name}
                </div>
                <div className="text-xs text-slate-500">Page {cite.page}<
              </div>
            </div>
          )}
        <p className="text-sm text-slate-600 italic line-clamp-3">

```

```

        "{cite.excerpt}"
      </p>
    </div>
  )})
</div>
</div>
</div>
)}

{/* Empty State */}
{!result && !loading && !error && (
  <div className="bg-slate-50 rounded-lg p-12 text-center border-2 border
    <ShieldCheck className="w-12 h-12 text-slate-400 mx-auto mb-3" />
    <p className="text-slate-600 text-sm">
      Ask a compliance question to get started
    </p>
  </div>
)}
</div>
);
}

```

4.4 Add Page

File: frontend/app/compliance/page.tsx

```

typescript

import { Layout } from "@app/components/Layout";
import NCUAComplianceCopilot from "@app/components/NCUAComplianceCopilot";

export default function CompliancePage() {
  return (
    <Layout>
      <div className="min-h-screen bg-gradient-to-br from-slate-50 to-slate-100
        <div className="max-w-6xl mx-auto">
          <NCUAComplianceCopilot />
        </div>
      </div>
    </Layout>
  );
}

```

4.5 Update Navigation

File: `frontend/app/components/Layout.tsx` (update sidebar)

Add to navigation links:

```
typescript

{
  href: "/compliance",
  label: "Compliance",
  icon: <ShieldCheck className="w-5 h-5" />,
  isActive: pathname.startsWith("/compliance"),
}
```

PHASE 5: Testing & Validation (Day 5-6)

5.1 Backend Testing

File: `backend/tests/test_ncua_copilot.py`

```
python
```

```

import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_ncua_query_endpoint_exists():
    """Test endpoint is registered"""
    response = client.post("/ncua/copilot/query", json={"query": "test"})
    assert response.status_code != 404

def test_ncua_query_valid_response():
    """Test valid query returns proper response"""
    response = client.post(
        "/ncua/copilot/query",
        json={"query": "What are NCUA fair lending requirements?"}
    )

    if response.status_code == 200:
        data = response.json()
        assert "answer" in data
        assert "citations" in data
        assert "confidence_score" in data
        assert len(data["citations"]) > 0

def test_ncua_query_empty_query():
    """Test empty query handling"""
    response = client.post(
        "/ncua/copilot/query",
        json={"query": ""}
    )
    # Should handle gracefully
    assert response.status_code in [400, 422]

def test_ncua_citations_have_sources():
    """Test citations include required fields"""
    response = client.post(
        "/ncua/copilot/query",
        json={"query": "What is ECOA?"}
    )

    if response.status_code == 200:
        data = response.json()

```

```
for citation in data["citations"]:
    assert "document_name" in citation
    assert "page" in citation
    assert "excerpt" in citation
```

Run tests:

```
bash

cd backend
pytest tests/test_ncua_copilot.py -v
```

5.2 Frontend Testing

Manual testing checklist:

- ☐ Component renders without errors
- ☐ Input field accepts text
- ☐ Submit button works
- ☐ Loading spinner shows during API call
- ☐ Answer displays correctly
- ☐ Citations render properly
- ☐ Error messages display
- ☐ Empty state shows
- ☐ Mobile responsive

5.3 Example Queries to Test

1. "What are the NCUA model risk requirements for machine learning?"
2. "Can we use churn predictions to restrict credit?"
3. "What does NCUA say about fair lending?"
4. "Are there NCUA rules for alternative data?"
5. "What's NCUA guidance on AI governance?"

5. Deployment Checklist

Pre-Deployment

- ☐ NCUA documents downloaded and placed in `backend/ncua_docs/`
- ☐ MongoDB vector index created in Atlas
- ☐ Environment variables configured (both backend and frontend)
- ☐ OpenAI API key active and has sufficient credits
- ☐ All tests passing

☐ Document ingestion script runs successfully

Deployment Steps

1. Backend Deployment (Render)

```
bash

# Push code to GitHub
git add .
git commit -m "Add NCUA Compliance Copilot"
git push origin main

# Render auto-deploys from GitHub
# Set environment variables in Render dashboard:
# - OPENAI_API_KEY
# - MONGODB_URL
# - MONGODB_DB_NAME
# - OPENAI_EMBEDDING_MODEL
# - OPENAI_LLM_MODEL
```

2. Run Document Ingestion

```
bash

# SSH into Render backend or run locally, then run:
python backend/app/ingest_ncua.py

# Verify in MongoDB Atlas:
# Collection ncua_vectors should have 150+ documents
```

3. Frontend Deployment (Vercel)

```
bash

# Vercel auto-deploys from GitHub
# Set environment variable:
# - NEXT_PUBLIC_API_URL=<your-render-backend-url>
```

4. Verify Deployment

```
bash
```

```
# Test endpoint:
curl -X POST https://<your-backend>.onrender.com/ncua/copilot/query \
  -H "Content-Type: application/json" \
  -d '{"query": "What is NCUA?"}'

# Should return: { "answer": "...", "citations": [...], ... }
```

6. File Structure

```
backend/
├─ app/
│   ├─ ingest_ncua.py (NEW)
│   ├─ models.py (UPDATED - add ComplianceQueryRequest, etc.)
│   ├─ routes.py (UPDATED - add /ncua/copilot/query endpoint)
│   └─ main.py (UPDATED - register router)
├─ ncua_docs/ (NEW - folder for PDFs)
│   ├─ 19-CU-04_Alternative_Data.pdf
│   ├─ 23-CU-01_Risk_Priorities.pdf
│   ├─ 12CFR701.pdf
│   ├─ RegulationB_Fair_Lending.pdf
│   └─ ModelRisk_Governance.pdf
├─ tests/
│   └─ test_ncua_copilot.py (NEW)
└─ requirements.txt (UPDATED)

frontend/
├─ app/
│   ├─ components/
│   │   └─ NCUAComplianceCopilot.tsx (NEW)
│   │   └─ Layout.tsx (UPDATED - add nav link)
│   └─ lib/
│       └─ api.ts (UPDATED - add queryNCUACopilot)
│   └─ types/
│       └─ index.ts (UPDATED - add Citation, ComplianceResponse)
│   └─ compliance/
│       └─ page.tsx (NEW)
└─ package.json (no changes needed)
```

7. Timeline

Phase	Task	Duration	Status
1	Setup & Configuration	1-2 days	
2	Document Ingestion Pipeline	1-2 days	
3	Backend API	1-2 days	
4	Frontend Component	1-2 days	
5	Testing & Validation	1 day	
TOTAL	Complete Feature	5-7 days	

8. Success Criteria

- ✓ Ingestion script successfully processes NCUA PDFs (150+ chunks)
- ✓ `/ncua/copilot/query` endpoint returns grounded answers with citations
- ✓ Frontend component loads without errors
- ✓ User can enter query and receive compliance answer
- ✓ Citations include document name, page, and excerpt
- ✓ Confidence scores display
- ✓ Error handling works (no docs found, API errors, etc.)
- ✓ Mobile responsive
- ✓ Tests pass
- ✓ Deployed to Render + Vercel

9. Example: End-to-End Flow

User enters query:

"Can we use machine learning to automatically deny credit applications? What does NCUA say?"

System processes:

1. Embeds query to vector
2. Searches MongoDB for similar NCUA documents
3. Retrieves 4 most relevant chunks
4. Passes to GPT-4o with regulatory context
5. LLM generates compliant answer

User sees response:

Compliance Officer Insight:
Using machine learning models to make credit decisions is permissible under NCUA oversight, but requires strict compliance protocols:

- 1. **Adverse Action Disclosure**: Automatic denials based on model scores constitute adverse actions. Members must receive clear notice of decision reasons per Regulation B.
- 2. **Model Validation & Bias Testing**: NCUA Letter 23-CU-01 requires independent validation of model performance across demographic groups to prevent disparate impact.
- 3. **Documentation**: Maintain audit trail showing model inputs, scores, and decision rationale for each application.

-  Regulatory Sources:
- NCUA Letter 19-CU-04 (Page 2) - Use of Alternative Data & Automated Systems
 - 12 CFR Part 1002 (Page 14) - Regulation B: ECOA Requirements
 - NCUA Supervisory Letter 23-EVAL-01 (Page 5) - Model Risk Governance

Confidence: 95%
Query ID: 550e8400-e29b-41d4-a716-446655440000

10. Estimated Costs

Item	Monthly Cost
OpenAI Embeddings (191 chunks, ~\$0.02 per 1M)	< \$0.01
OpenAI GPT-4o (assuming 100 queries/month)	~\$2-5
MongoDB Vector Search (Atlas free tier)	\$0
Render/Vercel Hosting	\$0
Total	~\$2-5/month

11. Next Steps (Recommended Enhancements)

After core feature is live:

- Add regulatory alert system (monitor NCUA for new guidance)
 - Implement query feedback loop (staff rates answer quality)
 - Add batch compliance checking for multiple members
 - Build audit report generator (compliance summary for examiners)
 - Create model card documenting RAG system performance
 - Add member-context aware queries (fair lending checks)
-

Document Version: 1.0

Last Updated: August 2026

Status: Ready for Implementation